

Индексы в MySQL

Что такое индексы?

Индексы в MySQL — это специальные структуры данных, которые улучшают скорость поиска и извлечения данных из таблиц. Они работают аналогично указателям в книге, позволяя быстро находить нужную информацию без необходимости просматривать всю таблицу.

Зачем нужны индексы?

- **Ускорение запросов:** индексы значительно ускоряют выполнение запросов SELECT.
- **Оптимизация сортировки:** индексы могут ускорить операции ORDER BY.
- **Улучшение производительности соединений:** индексы ускоряют операции JOIN.
- **Обеспечение уникальности:** индексы типа UNIQUE гарантируют уникальность значений в столбце.

Типы индексов в MySQL

PRIMARY KEY

Первичный ключ — это уникальный индекс, который идентифицирует каждую запись в таблице. Таблица может иметь только один первичный ключ.

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(50) NOT NULL,  
  email VARCHAR(100) NOT NULL  
);
```

Или добавление первичного ключа к существующей таблице:

```
ALTER TABLE users  
ADD PRIMARY KEY (id);
```

UNIQUE

Уникальный индекс гарантирует, что все значения в индексируемом столбце уникальны.

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(50) NOT NULL,  
  email VARCHAR(100) NOT NULL,  
  phone VARCHAR(20) UNIQUE NOT NULL  
);
```

```
    UNIQUE KEY unique_email (email)
);
```

Или добавление уникального индекса к существующей таблице:

```
ALTER TABLE users
ADD UNIQUE KEY unique_email (email);
```

INDEX

Обычный индекс ускоряет запросы, но не гарантирует уникальность значений.

```
CREATE TABLE products (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    category_id INT NOT NULL,
    INDEX idx_category (category_id)
);
```

Или добавление обычного индекса к существующей таблице:

```
CREATE INDEX idx_category ON products (category_id);
```

FULLTEXT

Полнотекстовый индекс используется для полнотекстового поиска в текстовых полях.

```
CREATE TABLE articles (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    content TEXT NOT NULL,
    FULLTEXT KEY ft_content (content)
);
```

Или добавление полнотекстового индекса к существующей таблице:

```
ALTER TABLE articles
ADD FULLTEXT KEY ft_content (content);
```

Использование полнотекстового поиска:

```
SELECT * FROM articles
WHERE MATCH(content) AGAINST('поисковый запрос' IN NATURAL LANGUAGE MODE);
```

SPATIAL

Пространственный индекс используется для индексации пространственных данных (геометрических типов).

```
CREATE TABLE locations (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    location POINT NOT NULL,
    SPATIAL INDEX sp_location (location)
);
```

Составные индексы

Составной индекс создается на основе нескольких столбцов.

```
CREATE TABLE orders (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    product_id INT NOT NULL,
    order_date DATE NOT NULL,
    INDEX idx_user_product (user_id, product_id)
);
```

Или добавление составного индекса к существующей таблице:

```
CREATE INDEX idx_user_product ON orders (user_id, product_id);
```

Создание и удаление индексов

Создание индекса

```
CREATE INDEX index_name ON table_name (column_name);
```

Создание составного индекса

```
CREATE INDEX index_name ON table_name (column1, column2, ...);
```

Удаление индекса

```
DROP INDEX index_name ON table_name;
```

Удаление первичного ключа

```
ALTER TABLE table_name DROP PRIMARY KEY;
```

Просмотр информации об индексах

Просмотр индексов таблицы

```
SHOW INDEX FROM table_name;
```

Просмотр создания таблицы с индексами

```
SHOW CREATE TABLE table_name;
```

Оптимизация индексов

Выбор правильных столбцов для индексирования

- Индексируйте столбцы, которые часто используются в условиях WHERE, JOIN и ORDER BY.
- Избегайте индексирования столбцов с низкой кардинальностью (малым количеством уникальных значений).
- Учитывайте, что индексы занимают дополнительное место на диске и замедляют операции вставки, обновления и удаления.

Порядок столбцов в составных индексах

Порядок столбцов в составном индексе имеет значение. Столбцы с высокой селективностью (большим количеством уникальных значений) должны идти первыми.

Использование EXPLAIN для анализа запросов

```
EXPLAIN SELECT * FROM users WHERE email = 'example@example.com';
```

Результат EXPLAIN показывает, какие индексы используются при выполнении запроса и как MySQL планирует выполнить запрос.

Ограничения и недостатки индексов

- **Дополнительное использование дискового пространства:** индексы занимают дополнительное место на диске.
- **Замедление операций модификации данных:** индексы замедляют операции INSERT, UPDATE и DELETE, так как требуют обновления индексных структур.
- **Ограничение на количество индексов:** MySQL имеет ограничение на количество индексов для таблицы (обычно 64).

Заключение

Индексы являются важным инструментом оптимизации производительности баз данных MySQL. Правильное использование индексов может значительно ускорить выполнение запросов, особенно для больших таблиц. Однако важно помнить о компромиссах между скоростью чтения и скоростью записи, а также о дополнительном использовании дискового пространства.